

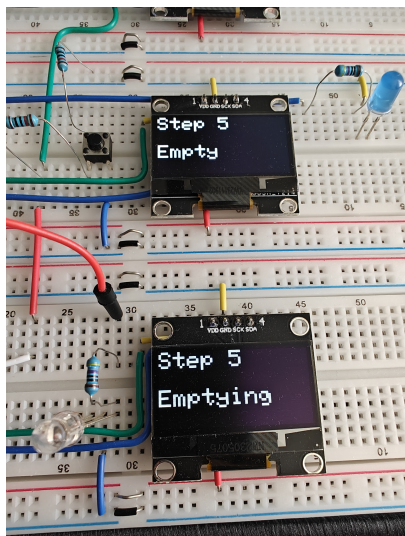


Figura 46: Step 5, Vaciar el contenido del tanque

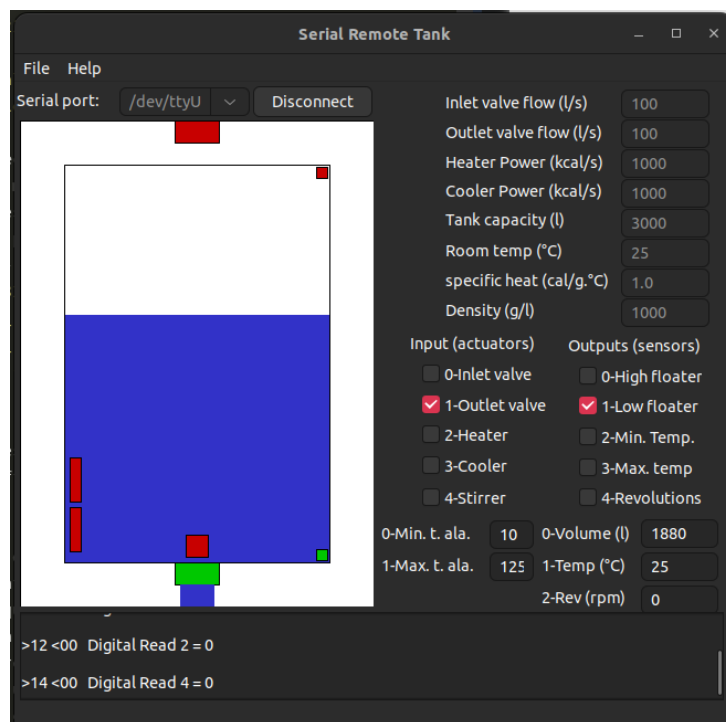


Figura 47: Step 5, Válvula de vaciado abierta

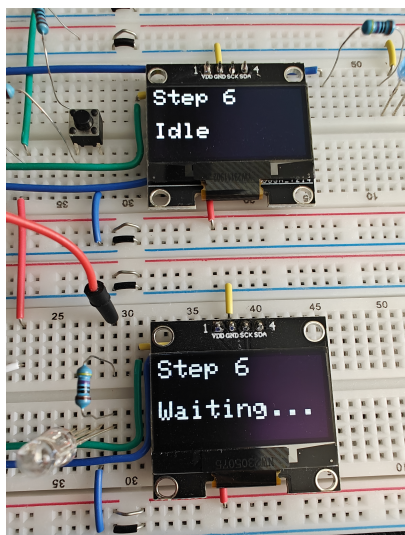


Figura 48: Step 6, Esperar la pulsación del botón para comenzar un nuevo proceso

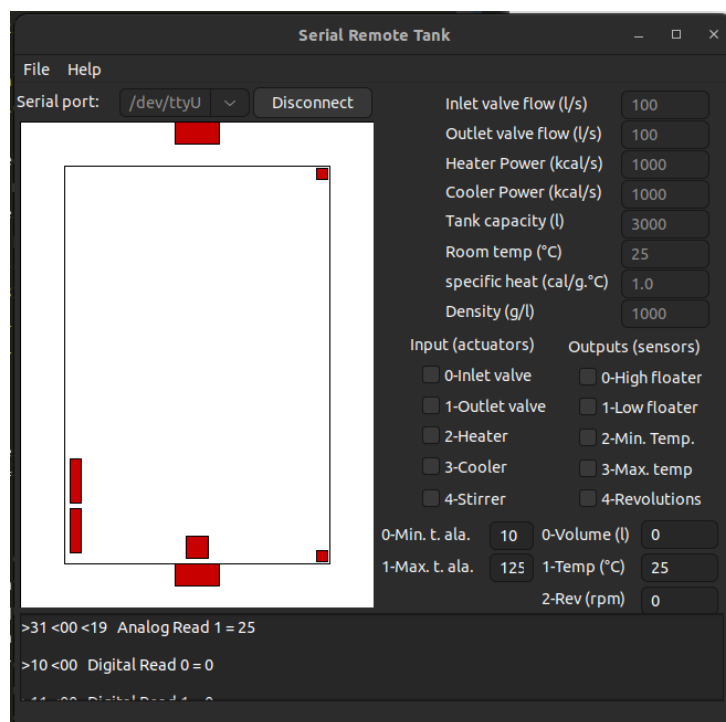


Figura 49: Step 6, Tanque en reposo

Este proceso simulado de saponificación permite probar y validar el funcionamiento del sistema de control diseñado, asegurando que cada paso se realiza con precisión y en el orden correcto. La sincronización y control preciso de cada etapa es crucial para obtener un producto de alta calidad y garantizar la eficiencia del proceso.

Cada step realizado en el sistema es revisado por el algoritmo de chequeo, el cual es implementado en los controladores para asegurar que el tanque se encuentra en el estado deseado antes de pasar al siguiente paso. Dependiendo del momento en el que se realice la comprobación, los requisitos serán unos u otros. A continuación se listan las condiciones exigidas al tanque tras cada paso del proceso químico completado:

- Checking Step 0: En este paso se comprueba que el volumen sea mayor o igual que 1485 litros, menor a 1515 litros (se ha dado un margen de $\pm 1\%$ en esta fase de llenado), que el volumen registrado en la iteración anterior sea igual que el actual (para asegurar que la válvula de llenado se haya cerrado correctamente) y que el flag de la boya de mínimo volumen esté activa.
- Checking Step 1: Por su parte, la comprobación del step 1 se basa en asegurar que la temperatura sea de al menos 34°C y de que la temperatura registrada anteriormente sea igual o menor a la actual, (para asegurar que el calefactor se ha detenido correctamente y, también se debe tener en cuenta que la temperatura de la mezcla tiende a disminuir ya que es mayor a la temperatura de la sala).
- Checking Step 2: En el chequeo del step 2 se comprueba que el volumen del tanque sea de 3000 litros y que los flags de las boyas de mínimo y máximo volumen estén activadas.
- Checking Step 3: Se revisa que la temperatura sea mayor o igual a 49°C y que el flag de las revoluciones esté desactivado.
- Checking Step 4: La cuarta comprobación se cerciora de que la temperatura sea menor o igual a 26°C , que la temperatura anterior sea igual que la actual (para asegurar que el enfriador esté desactivado) y que el flag de mínima temperatura no esté activado.
- Checking Step 5: El último chequeo debe confirmar que el tanque tenga un volumen de cero litros y que el flag de la boya de mínimo volumen esté desactivado.

El programa Serial Remote Tank (SR Tank), es una utilidad diseñada para simular un tanque de control remoto utilizando comunicación serial.

El tanque tiene varios sensores y actuadores que pueden ser leídos y controlados usando el protocolo de comunicación. Los parámetros del puerto de comunicación serial deben ser 19200 8N1.

Este software puede ser controlado desde un microcontrolador real e incluso virtual y, además, durante la simulación puede apreciarse como el tanque se llena, vacía, caliente, enfría o se agita en tiempo real gracias a una animación.

5.5.2. Forzar error por software

Se han incluido dos fallos por software en el código, específicamente diseñados para que en dos iteraciones concretas del bucle principal se envíen dos comandos erróneos. Estos comandos generan una discrepancia, lo que hace que el comparador envíe un comando de reset al controlador estropeado, forzando su reinicio. Este procedimiento permite verificar que el controlador se reinicia correctamente y retoma el proceso en el STEP adecuado, tal como se ilustró en la Figura 32.

5.5.3. Reinicio manual de un controlador

Adicionalmente, se ha probado el reinicio manual de uno de los controladores durante su funcionamiento. Al reiniciar manualmente el controlador, se genera una discrepancia, y el sistema debe recuperarse haciendo que el controlador reiniciado vuelva al STEP correspondiente del proceso en el momento actual. Esta prueba ha confirmado que el sistema puede gestionar el reinicio manual de los controladores sin interrumpir el flujo normal de las operaciones.

5.5.4. Desconexión manual de un controlador

De manera similar a la prueba de reinicio manual, se ha realizado una prueba desconectando manualmente uno de los controladores y volviéndolo a conectar. Esta acción también genera una discrepancia, y el sistema se recupera asegurándose de que el controlador que se volvía a conectar retomara el STEP correspondiente del proceso en el momento actual. Esta prueba también ha demostrado que el sistema es capaz de manejar desconexiones temporales de los controladores, garantizando la continuidad y estabilidad del proceso. Por seguridad, si

la desconexión de uno de los controladores dura demasiado tiempo, el sistema al dejar de ser redundante se considera inseguro, por lo que se procede a abortar el proceso y avisar a un operario del problema.

6

Estudio de la viabilidad de la simulación software del sistema

Comenzar el desarrollo de un proyecto utilizando un simulador en lugar de hacerlo directamente en físico, puede ser la mejor opción para aquellos que están empezando a familiarizarse con el mundo de Arduino. Es por eso que al inicio del TFG se utilizó el simulador PICSimLab [1] para agilizar el trabajo. Este capítulo describe el trabajo realizado con la simulación.

6.1. Ventajas del uso de un simulador

Las ventajas que tiene entorno emulado sobre uno real se podrían resumir en el siguiente listado:

- Facilidad para conectar componentes: Con un simulador, la conexión de componentes es mucho más sencilla y rápida. No es necesario lidiar con el cableado físico; en su lugar, se indica qué pin del Arduino usa cada componente. Se consigue así un ahorro de tiempo, y reduce los posibles errores en las conexiones.
- Ambiente seguro que previene daños: Se proporciona un ambiente seguro para experimentar y probar diferentes configuraciones sin preocuparse por posibles daños, tanto en los componentes como en el ordenador conectado al propio Arduino a través de USB.
- Depuración simplificada: El simulador incluye un depurador integrado que facilita el desarrollo y la identificación de errores en el código que se está ejecutando en el Arduino.

Además, cuenta con un osciloscopio virtual que permite visualizar cualquier línea simplemente especificándola, mostrando la señal configurada y lista para ser revisada.

- Acceso a una gran variedad de componentes y ahorro de costes: Se reduce significativamente el coste inicial del proyecto, ya que no es necesario comprar todos los componentes desde el principio. Además, permite probar una amplia variedad de componentes y configuraciones, de modo que solo se compren los elementos necesarios una vez que se ha determinado la mejor configuración para el sistema.
- Rapidez en el desarrollo inicial y preparación para el entorno real: Se trabaja a un ritmo mayor al no tener que lidiar con infinitud de componentes por cada pequeña prueba que se quiera realizar, y trabajar primero en un simulador permite a los desarrolladores ganar confianza y experiencia antes de pasar al montaje físico del proyecto. De esta manera, cuando llegue el momento de dar el salto al entorno real, con toda esa experiencia previa, se realizará un montaje preciso y más cuidado.

6.2. Modificaciones

El simulador PICSimLab [1] cuenta con todos los integrantes necesarios para simular el sistema completo, (Arduinos, resistencias, LEDs, pantallas, botones), permite trabajar con varias instancias, es robusto y eficiente. Sin embargo, carece del componente más sencillo e importante, los cables, sin los cuales no se puede conectar el Arduino de la primera instancia, con el de la segunda instancia para formar así el protocolo de comunicación Custom SPI.

Debido a esto y dado que este software es open source y permite su edición, se procedió a la creación de un nuevo periférico en PICSimLab para conseguir emular el funcionamiento de un cable que permita conectar los Arduinos de distintas instancias. Para esta labor se abordó primero el uso de sockets TCP/IP y después, la memoria compartida.

6.2.1. Sockets TCP/IP

Para emular el funcionamiento de un cable y conectar Arduinos de distintas instancias en PICSimLab, se ha desarrollado un nuevo periférico utilizando sockets TCP/IP. Este periférico permite leer y encapsular la información de ocho pines de un Arduino, transmitirla a otra

instancia mediante sockets TCP/IP, desencapsular la información y transmitirla al Arduino receptor. El funcionamiento del periférico se puede entender mediante un ejemplo sencillo:

- Encapsulación y transmisión de datos: El Arduino A quiere encender un LED conectado al Arduino B. Para ello, A pone en alto uno de sus pines (por ejemplo, el pin D4). El valor '1' en el pin D4 es leído y encapsulado junto con los estados de otros tres pines (por ejemplo, D5, D6 y D7 en bajo). Esta información encapsulada se transmite mediante sockets TCP/IP a la instancia que contiene el Arduino B.
- Desencapsulación y aplicación de datos: La instancia del Arduino B recibe la información, la desencapsula y aplica los estados recibidos a sus pines correspondientes. Si A puso en alto el pin D4, entonces B también pondrá en alto su pin D4. Los estados de los otros pines (D5, D6 y D7 en bajo) también se aplicarán a B.
- Bidireccionalidad de los cables: Los “cables” emulados son bidireccionales. Si los pines D8, D9, D10 y D11 están configurados como salidas en B y como entradas en A, el proceso se realiza a la inversa. B lee los valores de sus pines de salida, los encapsula y los envía a A mediante sockets TCP/IP. El Arduino A desencapsula la información y aplica los estados recibidos a sus pines de entrada correspondientes.

Para que estos “cables” emulados respondan de manera similar a un cable físico real, es indispensable que la transmisión de datos se realice a altas frecuencias. Esto asegura que los cambios de estado en los pines se reflejen rápidamente en las instancias conectadas, minimizando el retardo y mejorando la precisión de la emulación.

6.2.2. Shared memory

Para emular el funcionamiento de un cable y conectar Arduinos de distintas instancias en PICSimLab utilizando memoria compartida, he desarrollado un nuevo periférico que permite a cada Arduino acceder a una zona de memoria compartida común. En cada iteración, los Arduinos actualizan los valores de sus pines de salida en esta memoria compartida y leen los valores de los pines de entrada actualizados por otros Arduinos. El funcionamiento del periférico se puede entender mediante el mismo ejemplo sencillo anterior:

- Actualización y lectura de datos: El Arduino A quiere encender un LED conectado al Arduino B. Para ello, A pone en alto uno de sus pines (por ejemplo, el pin D4). El valor '1' en el pin D4 es escrito en la zona de memoria compartida, junto con los estados de otros tres pines (por ejemplo, D5, D6 y D7 en bajo). Esta información es accesible para la instancia que contiene el Arduino B.
- Lectura de datos desde la memoria compartida: La instancia del Arduino B accede a la memoria compartida, lee los estados almacenados y los aplica a sus pines correspondientes. Si A puso en alto el pin D4, entonces B también pondrá en alto su pin D4. Los estados de los otros pines (D5, D6 y D7 en bajo) también se aplicarán a B.
- Bidireccionalidad de los cables: Los “cables” emulados son bidireccionales, por lo que ambos Arduino pueden leer y escribir en la shared memory. Entonces si en B los pines D8, D9, D10 y D11 están configurados como salidas y como entradas en A, el proceso se realiza a la inversa. B actualiza los valores de sus pines de salida en la memoria compartida, y A lee estos valores y los aplica a sus pines de entrada correspondientes.

Al igual que en la opción con sockets TCP/IP, para que estos “cables” emulados respondan de manera similar a un cable físico real, es necesario que la actualización de los valores en la memoria compartida se realice a altas frecuencias.

6.3. El nuevo periférico

6.3.1. Diseño

El diseño del periférico, para cualquiera de sus versiones (ya que el front-end es el mismo en ambos casos), se puede ver en las Figuras 50 y 51.

6.3.2. Pruebas

Se llevaron a cabo varias pruebas utilizando el nuevo periférico desarrollado con sockets TCP/IP para PICSimLab. Primero, se probó el ejemplo descrito anteriormente, en el cual el Arduino A encendía un LED conectado al Arduino B. Los resultados de esta prueba fueron satisfactorios, demostrando que el periférico era capaz de transmitir y recibir señales correctamente a través de los ocho pines emulados.

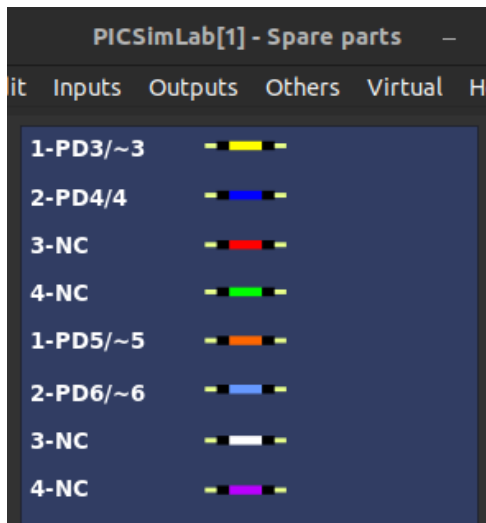


Figura 50: Nuevo periférico: cables entre instancias



Figura 51: Propiedades del periférico

Tras verificar la funcionalidad básica, se realizaron pruebas adicionales para confirmar que los ocho cables eran completamente funcionales. Estas pruebas incluyeron la activación y desactivación de múltiples pines de entrada y salida, asegurando que todos los pines podían transmitir y recibir señales adecuadamente.

Después, se procedió a probar la comunicación utilizando el protocolo "Custom SPI". Este protocolo requiere tres cables del Controlador al Comparador y uno del Comparador al Controlador. Sin embargo, los resultados indicaron que la tasa de refresco de las líneas por parte del periférico era insuficiente para captar los cambios rápidos en las líneas producidos por el protocolo. Aunque el periférico funcionaba correctamente para tareas simples, como encender o apagar un LED, no era adecuado para manejar un protocolo de comunicación tan rápido y exigente.

Debido a esto, se desarrolló también la versión con memoria compartida ya que, teóricamente, era más eficiente. Pero nada más lejos de la realidad, por un lado el periférico funcionaba mejor pero no lo suficiente como para lograr la comunicación.

6.3.3. Conclusiones

En conclusión, aunque el periférico demuestra ser funcional para la emulación básica de un cable, no es capaz de soportar las velocidades del protocolo debido a la necesidad de una mayor frecuencia de refresco para conseguir realizar la comunicación.

6.4. Limitaciones

Debido a estas limitaciones que presenta el periférico, se procedió a contactar con el creador del simulador con el fin de pedir consejo para hacer plenamente funcional al periférico. La conexión Málaga-Brasil fue fructífera y se consiguió configurar al proceso principal para realizar las actualizaciones del periférico más a menudo. El problema resultante fue el bloqueo del simulador entero debido a un refresco demasiado alto.

En resumen, si se configura el periférico con un refresco normal (cada 100 milisegundos), es insuficiente para la comunicación entre instancias, pero si se lleva a cabo un refresco a una frecuencia mayor, se bloquea el simulador completo, sin haber punto medio. Para concluir, se ha encontrado una limitación del propio simulador que hace inviable hacer funcionar un protocolo de comunicación sobre el periférico desarrollado. En un línea futura, se podría rediseñar el simulador para que acepte un mayor refresco del periférico y poder simular el sistema completo en PICSimLab.

Conclusiones y Líneas Futuras

7.1. Conclusiones

7.1.1. Eficacia del sistema de control redundante

La creación e implementación del sistema redundante de control basado en placas Arduino Nano ha demostrado su eficacia en garantizar la continuidad y estabilidad de un proceso crítico, como es controlar una vasija de reacción. Mediante numerosas pruebas, se ha corroborado que el sistema es capaz de identificar y corregir errores de manera eficaz, reduciendo al mínimo los períodos de inactividad y asegurando un funcionamiento confiable. Las capacidades de reinicio y sincronización de los controladores, junto con la habilidad para manejar discrepancias, son características esenciales en sistemas redundantes y han sido validados con éxito.

7.1.2. Uso del simulador

El uso del simulador PICSimLab [1] ha sido muy útil en las fases iniciales del desarrollo, ofreciendo un entorno seguro y controlado para poner a prueba y depurar el sistema. La posibilidad de simular conexiones y componentes sin riesgo de daños físicos ni la necesidad de tener todo el material, ha permitido experimentar de manera amplia y segura, acelerando el desarrollo.

7.1.3. Implementación de la interfaz gráfica

El desarrollo de una interfaz gráfica ha mejorado notablemente la visibilidad del sistema, dando la posibilidad a los usuarios de supervisar y gestionar el proceso en tiempo real. La interfaz simplifica la interacción con el sistema al ofrecer una representación visual clara del estado

de cada componente, lo cual permite abordar rápidamente cualquier irregularidad detectada.

7.2. Líneas futuras

7.2.1. Mejora del periférico en PICSimLab

La línea futura más importante es la mejora del código de PICSimLab para aumentar la velocidad de funcionamiento del periférico desarrollado. Actualmente, la tasa de refresco de las líneas es insuficiente para soportar el protocolo Custom SPI, lo que restringe su funcionalidad en simulaciones complejas como esta. La modificación del código de PICSimLab permitiría al periférico funcionar a una velocidad adecuada sin afectar el rendimiento global del simulador.

Esta mejora facilitaría la integración de todo el sistema al entorno simulado, conectando tres instancias de PICSimLab a través del periférico desarrollado, consiguiendo representar el sistema completo. Con este avance, se podría realizar una implementación del sistema en un entorno totalmente virtual, reduciendo la necesidad de hardware físico y permitiendo experimentar de manera más flexible y segura.

8

Referencias

8.1. Listado de referencias

- [1] PICSimLab Documentation. Disponible en <https://github.com/lcgamboa/picsimlab>
- [2] ISO C++ Standard. Disponible en <https://isocpp.org/std/the-standard>
- [3] Arduino Libraries Reference. Disponible en <https://www.arduino.cc/reference/en/libraries/>
- [4] Adafruit Learning System. Disponible en <https://learn.adafruit.com/>
- [5] PAGE Documentation. Disponible en <https://page.sourceforge.net/html/index.html>
- [6] Python Software Foundation. Python Documentation. Disponible en <https://docs.python.org/3/>
- [7] Vega Chemical Industry. Disponible en <https://www.vega.com/en-es/industries/chemical>
- [8] Motorola Inc. SPI Block Guide V03.06. Disponible en <https://www.nxp.com/docs/en/application-note/AN3904.pdf>
- [9] Analog Devices. Introduction to SPI Interface. Disponible en <https://www.analog.com/en/analog-dialogue/articles/introduction-to-spi-interface.html>
- [10] Circuit Basics. Basics of the SPI Communication Protocol. Disponible en <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>
- [11] Microchip Technology. Section 23. Serial Peripheral Interface (SPI). Disponible en <https://ww1.microchip.com/downloads/en/DeviceDoc/60001156G.pdf>
- [12] The Engineering Projects Blog. Disponible en <https://www.theengineeringprojects.com/blog/>
- [13] Serial Buses. FIRST Robotics Competition documentation. Disponible en <https://docs.wpilib.org/en/stable/docs/software/hardware/serial-buses.html>

- [14] Asynchronous Serial Communication: The Basics. ITP Physical Computing. Disponible en <https://itp.nyu.edu/physcomp/lessons>
- [15] Serial Communications Protocols - I2C. Altium Resources. Disponible en <https://resources.altium.com>
- [16] Texas Instruments. SPRUGP2A - Technical Reference Manual. Disponible en <https://www.ti.com/lit/ug/sprugp2a/sprugp2a.pdf?ts=1705984894791>
- [17] Kevin Dolan. Top Microcontrollers in 2023. kjdElectronics. Disponible en <https://kjdelectronics.com>
- [18] Gian Marco Iodice. TinyML Cookbook: Combine Machine Learning with Microcontrollers. O'Reilly Media, 2023.
- [19] Embedded World 2023: Texas Instruments Releases New ARM Cortex-M0 Plus Microcontroller Family. Electromaker. Disponible en <https://www.electromaker.io/blog>
- [20] Kristina Moyes. Best Microcontroller for IoT and How to Choose 2023. EmbedIc. Disponible en <https://www.embedic.com>
- [21] Adeel Javed. Building Arduino Projects for the Internet of Things. Apress, 2016.
- [22] Yury Magda. Radio Frequency Communications in Arduino Projects. CreateSpace Independent Publishing Platform, 2017.
- [23] Daniel Situnayake. TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2020.
- [24] J.M Hughes. Arduino: A Technical Reference. O'Reilly Media, 2016.
- [25] Simon Monk. 30 Arduino Projects for the Evil Genius. McGraw-Hill Education, 2010.
- [26] Gregory T. Long and Matthew D. Stewart. "Redundant Systems: A Comprehensive Overview." IEEE Transactions on Reliability, 2020. Disponible en <https://ieeexplore.ieee.org/document/9145697>
- [27] IEEE Xplore. "Optimal Design of Hybrid Redundant Systems With Delayed Failure-Driven Standby Mode Transfers." Disponible en <https://ieeexplore.ieee.org/document/7046392>

- [28] IEEE Xplore. “Reliable Redundant Control Design for Nonlinear Uncertain Systems.”
Disponible en <https://ieeexplore.ieee.org/document/1239071>
- [29] Axelson, J. (2007). “Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports
for Embedded Systems”. Lakeview Research. Disponible en <https://www.8051projects.net>
- [30] Tektronix. TBS1000 Series User Manual. Disponible en <https://download.tek.com/manual/TBS1000C-Series-Digital-Storage-Oscilloscope-User-Manual-077157100.pdf>

Apéndice A

Librería SRTank.h y Datasheets

A.1. Librería SRTank

La librería SRTank . h ha sido desarrollada para el uso de forma sencilla y eficaz del *Serial Remote Tank* del simulador PICSimLab [1].

Funciones de la Librería SRTank . h

A continuación se presenta el listado de las funciones disponibles:

- **Getters para los outputs digitales:**

- `unsigned char getHighFloater()`
- `unsigned char getLowFloater()`
- `unsigned char getMaxTemperature()`
- `unsigned char getMinTemperature()`

- **Getters para los outputs analógicos:**

- `unsigned int getVolume()`
- `unsigned int getTemperature()`

- **Setters para los outputs digitales:**

- `void setVolume(void)`
- `void setTemperature(void)`
- `void setHighFloater(void)`
- `void setLowFloater(void)`

- `void setMaxTemperature(void)`
- `void setMinTemperature(void)`

■ **Funciones con outputs digitales:**

- `void openInletValve(void)`
- `void closeInletValve(void)`
- `void openOutletValve(void)`
- `void closeOutletValve(void)`
- `void turnOnHeater(void)`
- `void turnOffHeater(void)`
- `void turnOnCooler(void)`
- `void turnOffCooler(void)`
- `void turnOnAgitator(void)`
- `void turnOffAgitator(void)`

■ **Setters para los outputs analógicos:**

- `void setMaxTemperatureAlarm(unsigned int temperature)`
- `void setMinTemperatureAlarm(unsigned int temperature)`

■ **Funciones complejas:**

- `void fillTank(void)`
- `void fillTank(unsigned int volume)`
- `void emptyTank(void)`
- `void emptyTank(unsigned int volume)`
- `void heatTank(void)`
- `void heatTank(unsigned int temperature)`
- `void coolTank(void)`
- `void coolTank(unsigned int temperature)`

- `void stirTank(void)`
- `void stirTank(unsigned int time)`

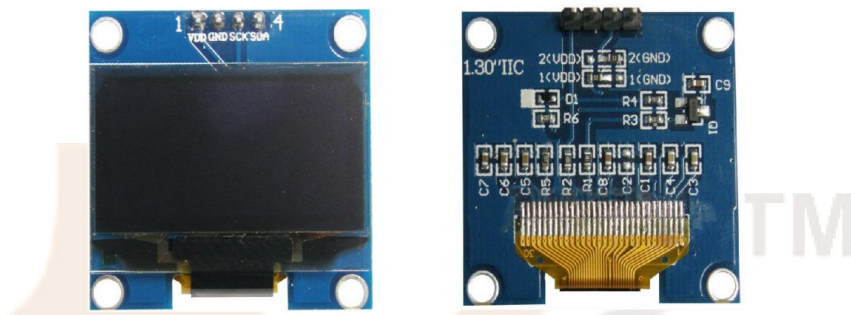
■ **Funciones relacionadas con el tanque:**

- `SRTankData getInit(void)`
- `SRTankData2 getInit2(void)`
- `SRTankData getStructure(void)`
- `SRTankData2 getStructure2(void)`
- `void setStructure(void)`

A.2. LCD OLED 1.3" DataSheet

RG Rajguru Electronics (I) Pvt. Ltd.

OLED 4 pin 128x64 Display Module 1.3" Blue Color



In contrast to LCD technology, Organic Light-Emitting Diode (OLED) displays do not require a backlight and are regarded as the ultimate technology for the next generation of flat-panel displays.

OLED displays are composed of a thin, multi-layered organic film placed between an anode and cathode, which are made up of electric conductive transparent Indium Tin Oxide.

The multi-layered organic film includes a Hole Transporting Layer, Emission Layer and Electron Transporting Layer.

By applying an appropriate electrical voltage, the holes and electrons are injected into the Emission Layer from the anode and cathode respectively and combine to form excitons, after which electroluminescence occurs.

This 1.3" 128*64 Blue OLED Module offers 128*64-pixel resolution. They are featuring much less thickness than LCD Displays with good brightness and produce better and true colors.

This OLED Display Module is very compact and will add a great ever user interface experience to your Arduino project. The connection of this display with Arduino is made through the I2C (also called as IIC) serial interface.

The 1.3" 128*64 Blue OLED Display Module produces blue text on black background with very good contrast when supplied with 3.3V-5V Supply. The OLED Display Modules also offers a very wide viewing angle. Total Current when running together is 8 mA.

FEATURES:

- Supply voltage: 3.3V-5V
- Pixel: 128*64
- Display size- 1.3 inch
- Operating temperature range: -40°C - +80°C
- Use I2C Interface
- Chip No: SSD1306
- Color: Blue
- Super high contrast and brightness(adjustable)

- PCB Size: 33.7 mm x 35.5 mm
- Low power consumption
- High contrast, thus supporting clear display with no need of backlight
- For OLED SSD1306, a more elaborate and beautiful screen than LCD with more functions

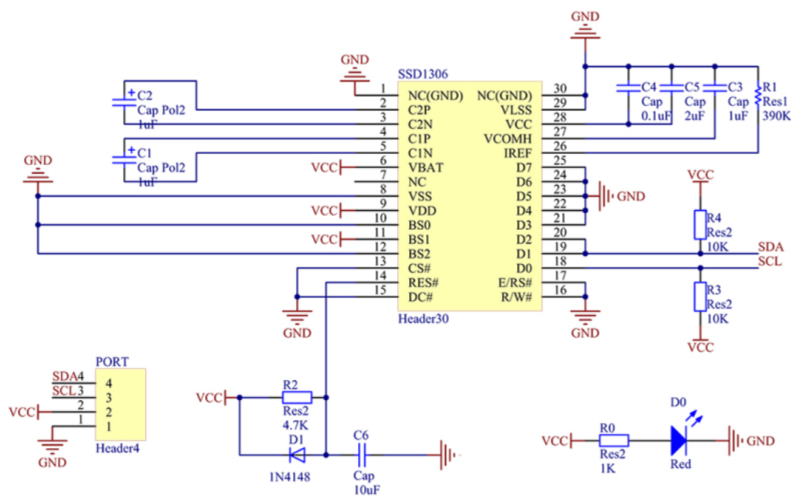
PIN DESCRIPTION:

Pin No.	Pin Name	Description
1.	Supply Voltage (Vcc, 5V)	Can be powered by either 3.3V or 5V
2.	Ground (GND)	Pin Ground
3.	Serial Clock(SCL)	Pin SCL of I2C interface
4.	Serial Data(SDA)	Pin SDA of I2C interface

ELECTRICAL CHARACTERISTICS:

ITEM	SYMBOL	TEST CONDITION	MIN	TYP	MAX	UNIT
Operating voltage	VDD	Ta=25°C	-----	5.0	-----	V
Operating voltage for LCD	VLCD	Ta=25°C	-----	5.0	-----	V
Supply current	IDD	Ta=25°C, VDD=5.0V	-----	2.0	3.0	mA
Supply current for Back light	IF	Ta=25°C, VF=5V	-----	20	-----	mA

SCHEMATIC DIAGRAM:



TEST:

To use the OLED with Arduino, you can take the following steps.

Step1:Connecting circuit

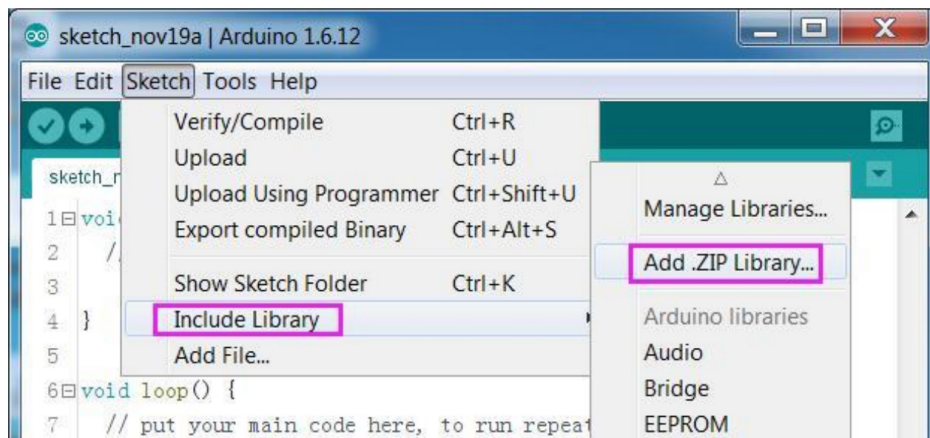
SSD1306 ----- Arduino Uno/Mega2560
GND ----- GND
VCC ----- 5V
SCL ----- A5 Uno/pin21 Mega2560
SDA ----- A4 Uno/pin20 Mega2560

Step2: Install the Arduino IDE in your computer. For detailed steps, refer to

http://wiki.sunfounder.cc/index.php?title=Install_Arduino_Software

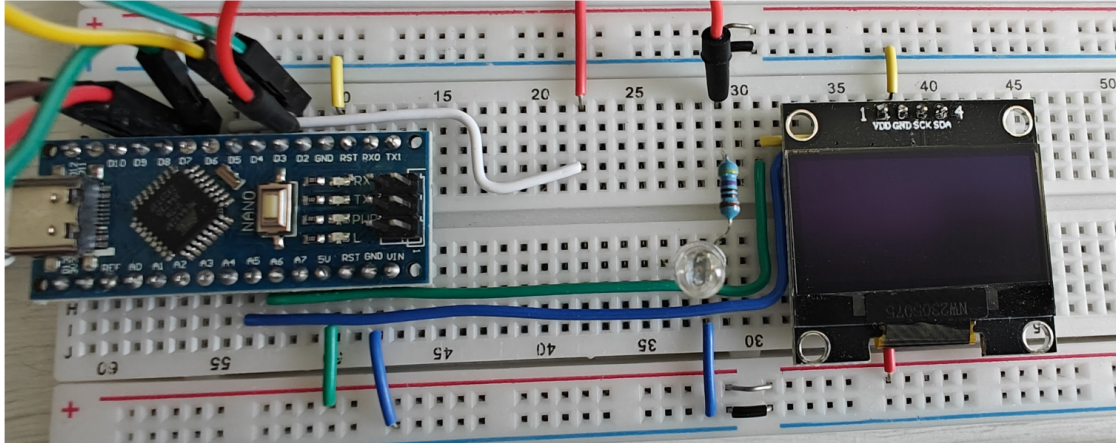
Step3: Add U8glib library.

- 1) Through the links at the bottom of the page to **download** [U8glib.zip](#) and unzip.
- 2) the Arduino IDE, import the library from the "Add Library" Menu.

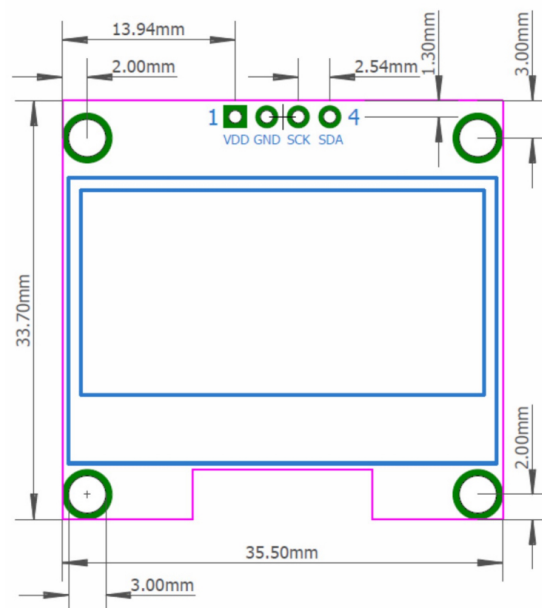


- 3) Find the **u8glib** folder you just extracted click **open**.

Step4 : Open **Graphics_Test. Ino** , Before clicking the Upload icon, set the **Port** and **Board** under **Tools** menu. Then upload the code to the board



OUTER DIMENSION:



A.3. 8 LEDs ring DataSheet

A.3.1. LED WS2812B



WS2812B

Intelligent control LED
integrated light source

Features and Benefits

- Intelligent reverse connect protection, the power supply reverse connection does not damage the IC.
- The control circuit and the LED share the only power source.
- Control circuit and RGB chip are integrated in a package of 5050 components, form a complete control of pixel point.
- Built-in signal reshaping circuit, after wave reshaping to the next driver, ensure wave-form distortion not accumulate.
- Built-in electric reset circuit and power lost reset circuit.
- Each pixel of the three primary color can achieve 256 brightness display, completed 16777216 color full color display, and scan frequency not less than 400Hz/s.
- Cascading port transmission signal by single line.
- Any two point the distance more than 5m transmission signal without any increase circuit.
- When the refresh rate is 30fps, cascade number are not less than 1024 points.
- Send data at speeds of 800Kbps.
- The color of the light were highly consistent, cost-effective..

Applications

- Full-color module, Full color soft lights a lamp strip.
- LED decorative lighting, Indoor/outdoor LED video irregular screen.

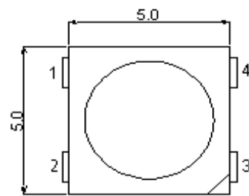
General description

WS2812B is a intelligent control LED light source that the control circuit and RGB chip are integrated in a package of 5050 components. It internal include intelligent digital port data latch and signal reshaping amplification drive circuit. Also include a precision internal oscillator and a 12V voltage programmable constant current control part, effectively ensuring the pixel point light color height consistent.

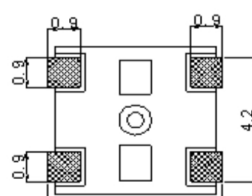
The data transfer protocol use single NZR communication mode. After the pixel power-on reset, the DIN port receive data from controller, the first pixel collect initial 24bit data then sent to the internal data latch, the other data which reshaping by the internal signal reshaping amplification circuit sent to the next cascade pixel through the DO port. After transmission for each pixel, the signal to reduce 24bit. pixel adopt auto reshaping transmit technology, making the pixel cascade number is not limited the signal transmission, only depend on the speed of signal transmission.

LED with low driving voltage, environmental protection and energy saving, high brightness, scattering angle is large, good consistency, low power, long life and other advantages. The control chip integrated in LED above becoming more simple circuit, small volume, convenient installation.

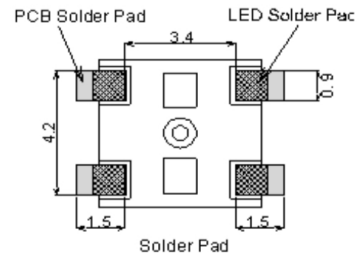
Mechanical Dimensions



Top View

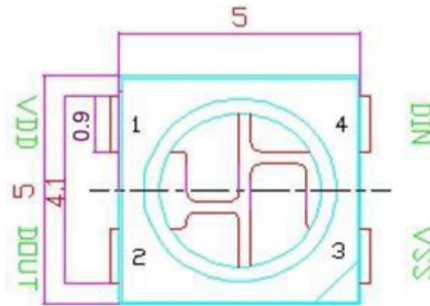


Back View



Solder Pad

PIN configuration



PIN function

NO.	Symbol	Function description
1	VDD	Power supply LED
2	DOUT	Control data signal output
3	VSS	Ground
4	DIN	Control data signal input

Absolute Maximum Ratings

Prameter	Symbol	Ratings	Unit
Power supply voltage	V _{DD}	+3.5~+5.3	V
Input voltage	V _I	-0.5~V _{DD} +0.5	V
Operation junction temperature	T _{opt}	-25~+80	°C
Storage temperature range	T _{stg}	-40~+105	°C

Electrical Characteristics (T_A=-20~+70°C, V_{DD}=4.5~5.5V, V_{SS}=0V, unless otherwise specified)



WS2812B

Intelligent control LED
integrated light source

Prameter	Smybol	conditions	Min	Tpy	Max	Unit
Input current	I_I	$V_I=V_{DD}/V_{SS}$	—	—	± 1	μA
Input voltage level	V_{IH}	D_{IN}, SET	$0.7V_{DD}$	—	—	V
	V_{IL}	D_{IN}, SET	—	—	$0.3 V_{DD}$	V
Hysteresis voltage	V_H	D_{IN}, SET	—	0.35	—	V

Switching characteristics ($T_A=-20\sim+70^{\circ}C$, $V_{DD}=4.5\sim5.5V$, $V_{SS}=0V$, unless otherwise specified)

Prameter	Symbol	Condition	Min	Tpy	Max	Unit
Transmission delay time	t_{PLZ}	$CL=15pF, D_{IN} \rightarrow D_{OUT}, R_L=10K\Omega$	—	—	300	ns
Fall time	t_{THZ}	$CL=300pF, OUTR/OUTG/OUTB$	—	—	120	μs
Input capcity	C_I	—	—	—	15	pF

RGB IC characteristic parameter

Emitting color	Model	Wavelength(nm)	Luminous intensity(mcd)	Voltage(V)
Red	13CBAUP	620-625	390-420	2.0-2.2
Green	13CGAUP	522-525	660-720	3.0-3.4
Blue	10R1MUX	465-467	180-200	3.0-3.4

Data transfer time ($TH+TL=1.25\mu s \pm 600ns$)

T0H	0 code ,high voltage time	0.4us	$\pm 150ns$
T1H	1 code ,high voltage time	0.8us	$\pm 150ns$
T0L	0 code , low voltage time	0.85us	$\pm 150ns$
T1L	1 code ,low voltage time	0.45us	$\pm 150ns$
RES	low voltage time	Above 50 μs	

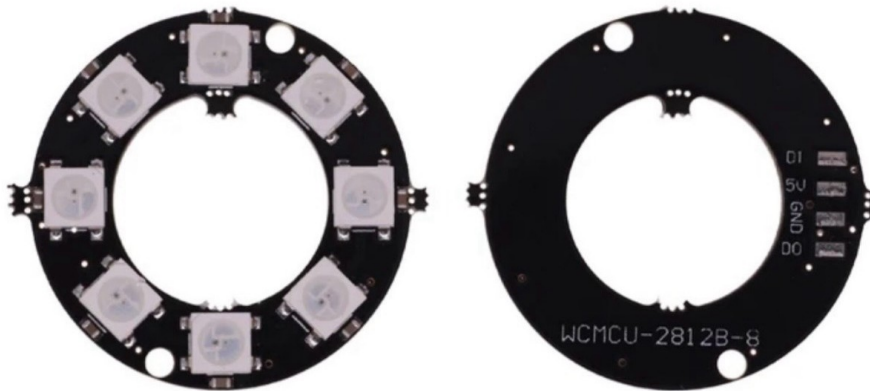
A.3.2. LEDs ring



Handson Technology

WS2812B RGB Neopixel LED Ring

These LED strips are just about the best way to get tons of colorful LED light with a minimum of wiring and fuss! Each ring is 3cm in diameter and contains 8 light elements. Each element contains a red, green, and blue LED, as well as a tiny WS2812 controller chip. This means that each element is individually addressable and can accept brightness values of 0-255 for each of its LEDs, giving you 24 bits of color precision overall. Each LED is activated by using a special serial data protocol. This protocol allows many LEDs to be daisy chained together so that one microcontroller can control the whole lot – with a single data signal. Any number of these LEDs can be chained together by connecting one module's data-out (DO) pin to another's data-in (DI) pin. This LED is compatible with Neopixel & FastLED Arduino Library.



Brief Data:

- Ring Diameter: Ø3cm.
- Ring of 8 pieces of 5050 Red, Green and Blue (RGB) LEDs.
- Each LED element is individually addressable with 24-bit color (8 bits each for R, G & B), 16 million colors.
- 3-Wires easy connection, 2-power supply lines and 1 data line.
- Interface: Serial (Shift Register).
- Compatible with Neopixel & FastLED Arduino Library.
- Data Speed: 800Kbps.
- Mounting Hole: 4-position M2 Screw.
- Power Supply: 5V.

Application Introducing to WS2812B LED Strip with Arduino:

This is the coolest type of LED strips in the market today. You can control the brightness and the color of each LED individually, which allows you to produce amazing and complex effects in a simple way. This LED strip is made by WS2812B LEDs wired in series. These LEDs have a IC built right into the LED. This allows a communication via a one-wire interface. This means that you can control lots of LEDs using just one digital pin of your Arduino. In the following figure you can see the chip inside the LED. The LED is an RGB LED and works like so.

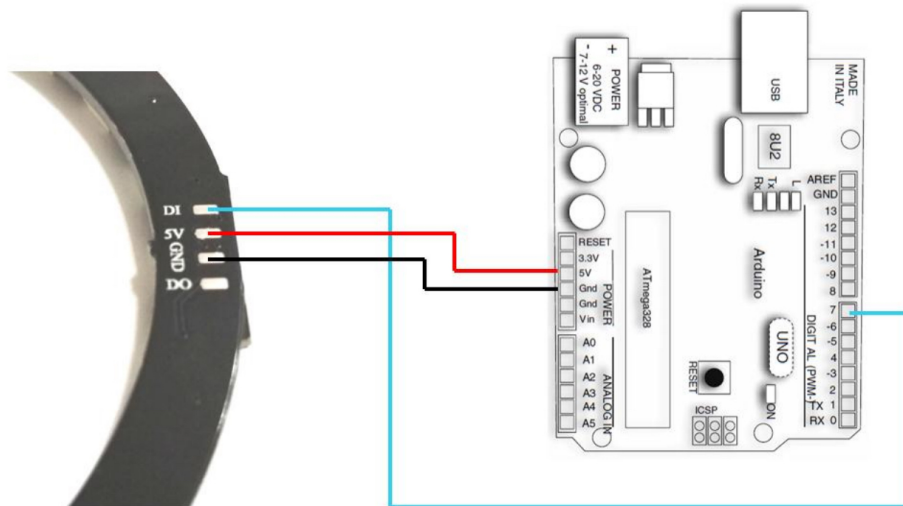


Powering the WS2812B LED Strip:

The LED ring should be powered using a 5V power source. At 5V, each LED draws about 50mA, when set to its full brightness. This means that for every 8 LEDs, the strip may draw as much as 1.2 A. Make sure you select a power source that matches the module's needs. If you end up using an external supply, don't forget to connect the power supply ground to the Arduino ground.

Arduino Schematics:

In this example, the WS2812B LED ring will be powered using the 5V from Arduino board. In this case, we are controlling 8-LEDs per ring. Bear in mind that if you want to control many LEDs, you'll need to use an external power supply.





UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga